SOFTWARE REQUIREMENTS SPECIFICATION

K O D B T W

Unified Coding Profile, Analytics & Contest Intelligence Platform

IEEE-style structure adapted to the implemented product

Document ID	KBTW-SRS-002
Version	2.0
Issue date	11 October 2026
Status	Implementation-aligned baseline
Product repository	github.com/sajalgoyal007/KodBTW
Frontend deployment	https://kodbtw.vercel.app
Backend deployment	https://kodbtw-backend.onrender.com
Baseline commit	90617732298391d84e258bf932e90cb59b96a106

Prepared from the repository's main-branch implementation and supplied SRS baseline.

This document describes observed and source-supported behavior. It does not claim formal IEEE certification or independent security certification.

Document control

Item	Description
Purpose	Define the current, evidence-based software requirements for KodBTW for review, testing, maintenance, and traceability.
Audience	Project team, evaluators, testers, maintainers, and stakeholders.
Baseline	GitHub main branch at commit 90617732298391d84e258bf932e90cb59b96a106, plus production UI observation of the contest calendar dated 11 October 2026.
Standard approach	Uses the familiar IEEE 830 SRS section flow as a structure. It is an adapted project SRS and does not assert formal IEEE compliance.
Evidence notation	Implemented = supported by source/configuration; Tested = tests/build are reported passing; Production observed = visible in the deployed application. Untested runtime claims are marked explicitly.

Revision history

Version	Date	Change
1.0	10 October 2026	Initial friend-provided SRS; described an older repository snapshot and preceded the contest calendar.
2.0	11 October 2026	Rewritten for current KodBTW architecture, integrations, authentication, contest calendar, deployments, scope limits, verification evidence and traceability.

Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features and Functional Requirements
5. Non-functional Requirements
6. Data Requirements
7. Security, Privacy and Compliance
8. Use Cases
9. Verification and Acceptance Criteria
10. Requirements Traceability
11. Constraints, Assumptions and Out of Scope
Appendix A. Deployment and Runtime Configuration
Appendix B. Acronyms and References

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the externally visible behavior, interfaces, constraints, and quality requirements for KodBTW. It is intended to support implementation review, test planning, acceptance checks, and maintenance. Requirements are written as “shall” statements where a testable expectation can be defined. Current implementation and verification state are identified separately to prevent a planned requirement from being mistaken for a delivered capability.

1.2 Scope

KodBTW is a web application that unifies supported coding-platform identities and selected statistics, persists sourced observations, presents dashboard and historical analytics, exposes leaderboard views, allows a user to share a public developer profile, and lists upcoming Codeforces contests. It is a single-user-facing product with an administrator-authorized leaderboard synchronization operation; it is not currently a campus administration or mentor-monitoring system.

1.3 Product objectives

- Provide one KodBTW account for managing a user profile and multiple coding-platform handles.
- Show available platform statistics with provenance and represent unsupported or missing metrics as unavailable rather than inventing values.
- Persist dated snapshots so history and analytics are traceable to collected observations.
- Provide owner-scoped dashboard, leaderboard and profile functions with token-based authentication.
- Show upcoming Codeforces contests, start times in the user’s local timezone, countdowns, filters, source status, and graceful empty/error states.

1.4 Definitions and abbreviations

Term	Definition
API	Application Programming Interface.
JWT	JSON Web Token used as a bearer authentication token.
Platform account	A user-owned record containing a platform identifier and public handle/profile URL.
Snapshot	A dated, persisted observation of metrics returned by a supported platform adapter.
Source tag	A label such as LEETCODE_REAL, CODEFORCES_REAL, or MOCK that identifies the origin/class of statistics.
TTL cache	Time-to-live cache that retains data for a configured period before refresh is required.
Contest calendar	Upcoming-contest list supplied by the Codeforces contest provider and cached by the backend.
Fixture mode	Deterministic local/test data source for testing contest-calendar scenarios without contacting Codeforces.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (historical structure reference; no claim of certification).
- Repository: https://github.com/sajalgoyal007/KodBTW
- docs/ARCHITECTURE.md, docs/CODEFORCES_INTEGRATION.md, docs/platform-data-acquisition.md and docs/contest-calendar-fixture-mode.md.
- Backend controllers, services, security configuration, adapter implementations, Flyway migrations and tests on main.
- User-supplied SRS.pdf, Version 1.0, used as a baseline and corrected where it no longer matched the current repository.

2. Overall Description

2.1 Product perspective

KodBTW is a monorepo with a React single-page frontend and a Spring Boot REST backend. The frontend communicates with the backend through HTTP APIs; the backend authenticates requests, enforces resource ownership, invokes platform adapters, persists supported data in MySQL through Spring Data JPA, and applies schema migrations through Flyway.

Layer	Current implementation
Frontend	React 18, TypeScript, Vite, React Router and a custom CSS design system.
Backend	Java 21, Spring Boot 3.3, Spring Security, Spring Data JPA/Hibernate, Maven.
Authentication	Stateless JWT bearer authentication; public registration, login, health, and public-profile reads.
Database	MySQL 8; schema evolution managed by ordered Flyway migrations; Hibernate configured to validate rather than mutate schema.
Platform statistics	LeetCode and Codeforces live-source adapters are documented. CodeChef, GeeksforGeeks and HackerRank are catalogued, but the current public README describes them as mock sources; they must not be represented as verified live integrations.
Contest feed	A Codeforces contest-list provider, in-memory TTL cache and stale-data fallback; fixture mode is available for deterministic scenarios.

2.2 Product functions

- Register, sign in and retrieve the authenticated user identity.
- Read and update the KodBTW profile.
- Create, retrieve, update, list and delete owned platform-account records.
- Request a sync for an owned account and inspect statistics and synchronization status.
- View dashboard totals, analytics, historical snapshots, insights, activity and persisted contest-intelligence summaries.
- View leaderboard pages, college-filtered results and personal rank; allow only an ADMIN authority to invoke manual leaderboard synchronization.

- Read an opt-in/public developer profile by username.
- View upcoming Codeforces contests with local-time display, countdown, a platform selector, refresh action and loading, empty, error or stale-data feedback.

2.3 User classes

User class	Capabilities
Visitor	Open landing page, register, log in, and view an available public developer profile.
Registered user	Manage own profile and platform accounts; access authenticated dashboard, analytics, contest, leaderboard and synchronization functions.
Administrator authority	Invoke the protected POST /api/leaderboard/sync operation. This is an authority on an authenticated principal, not a full campus administrator workflow.

2.4 Operating environment

- Backend runtime: Java 21; Spring Boot application packaged in a Docker image.
- Frontend build/runtime: Node.js 24.x was selected in the observed Vercel deployment; project documentation states Node.js 18+ and npm.
- Database: MySQL 8-compatible service configured through environment variables.
- Observed frontend production URL: https://kodbtw.vercel.app.
- Observed backend service URL: https://kodbtw-backend.onrender.com. Deployment configuration and runtime environment can change independently of source code.

2.5 Design and implementation constraints

- DB_URL, DB_USERNAME, DB_PASSWORD and JWT_SECRET are required runtime settings; secrets must not be committed or compiled into frontend assets.
- Production CORS must use exact trusted origins; wildcard origins are not supported for production.
- Missing metrics must remain null/unavailable when upstream sources do not return them.
- Upstream platform availability, rate limits, source contracts and terms affect sync reliability.
- The LeetCode integration uses a public but undocumented GraphQL endpoint; its stability and permission are not guaranteed by the implementation.
- Codeforces user.info does not expose all coding metrics (for example, solved counts); those metrics must not be fabricated.
- CodeChef, GeeksforGeeks and HackerRank support is limited by the version of the code and data sources actually configured. The current repository README labels them as MOCK sources.

3. External Interface Requirements

3.1 User interface

Route	Access	Purpose
/	Public	Landing / product entry page.
/login	Visitor-only	User authentication form.
/register	Visitor-only	Account creation form.

Route	Access	Purpose
/u/{username}	Public	Public developer profile page.
/dashboard	Authenticated	Personal platform summary and dashboard.
/analytics	Authenticated	Analytics and progress views.
/contests	Authenticated	Upcoming contest calendar and contest/platform standings.
/leaderboard	Authenticated	Leaderboard pages and personal ranking.
/profile	Authenticated	KodBTW profile management.
/coding-profiles	Authenticated	Connect and manage coding-platform accounts.

3.2 REST API interfaces

Method / path	Access	Expected behavior
POST /api/auth/register	Public	Validate registration request and create user account.
POST /api/auth/login	Public	Validate credentials and return JWT/user summary.
GET /api/auth/me	JWT required	Return current authenticated identity.
GET /api/health	Public	Return process-liveness response.
GET /api/profile; PUT /api/profile	JWT required	Read or upsert current user's profile.
GET /api/public/profiles/{username}	Public	Return sanitized public profile data.
GET /api/platform-accounts	JWT required	List caller-owned platform-account records.
POST /api/platform-accounts	JWT required	Create a platform-account record for caller.
GET/PUT/DELETE /api/platform-accounts/{id}	JWT required	Read, update or delete only a record owned by caller.
GET /api/platform-accounts/{id}/stats	JWT required	Return available stats for an owned account.
POST /api/platform-accounts/{id}/sync	JWT required	Request on-demand sync for an owned account, where live sync is supported.
GET /api/platform-accounts/{id}/sync-status	JWT required	Return the owned account's last attempt/success/failure status.
GET /api/dashboard/stats	JWT required	Return dashboard totals from eligible persisted data.
GET /api/dashboard/analytics	JWT required	Return analytics derived from persisted observations.
GET /api/dashboard/history?range=30d	JWT required	Return history for a supported range.
GET /api/dashboard/insights	JWT required	Return data-derived insights.
GET /api/dashboard/activity; /api/dashboard/contests	JWT required	Return activity and persisted-data contest intelligence.
GET /api/contests? platform=CODEFORCES&limit=50	JWT required	Return upcoming contest calendar; accepted platform filters are ALL/CODEFORCES and limit is 1–100.

Method / path	Access	Expected behavior
GET /api/leaderboard	JWT required	Return paginated global leaderboard with supported sort/filter options.
GET /api/leaderboard/college?college=...	JWT required	Return college-filtered leaderboard.
GET /api/leaderboard/me	JWT required	Return current user's ranking summary.
POST /api/leaderboard/sync	ADMIN authority	Run manual leaderboard synchronization.

3.3 External software interfaces

- MySQL: accessed through JDBC/JPA; Flyway executes versioned migrations on startup.
- Codeforces statistics: GET https://codeforces.com/api/user.info?handles={handle}; no KodBTW user credentials are needed. Unsupported metrics remain null.
- Codeforces contests: provider retrieves contest-list information, retains valid upcoming contests, sorts by start time, maps to official contest URLs and tags the source as CODEFORCES_API.
- LeetCode statistics: public GraphQL query is used by the live adapter; it is undocumented and subject to upstream change.
- No platform password, cookie or session token is required for the described public-profile integrations.

3.4 Communications and error interface

The frontend sends JSON API requests and uses a bearer JWT for protected operations. Login and registration requests are unauthenticated requests and must not attach a stale stored JWT. Backend errors should avoid exposing stack traces, exception internals, database details or credentials. Platform and contest-provider failures must be represented as categorized errors or a documented stale-data response rather than silently replacing real data with synthetic values.

4. System Features and Functional Requirements

Priority labels: P1 = core/release-critical, P2 = important supporting capability. "Current evidence" distinguishes source review, automated tests, and direct production observation.

ID	Feature	Requirement (shall)	Priority	Mapping	Evidence/status
FR-01	Account registration	The system shall validate a registration request and create a unique user account.	P1	AuthController, UserService; auth tests	Implemented; test suite evidence available from project history.
FR-02	Sign in	The system shall authenticate valid credentials and return a bearer JWT and user summary.	P1	AuthController, AuthService, JwtService	Implemented; login observed as a key production flow, but independent full security audit not performed.
FR-03	Current identity	The system shall return the identity associated with the valid JWT.	P1	GET /api/auth/me	Implemented in source.
FR-04	Profile management	The system shall allow a registered user to read and update their own profile.	P1	ProfileController, ProfileService	Implemented in source.

ID	Feature	Requirement (shall)	Priority	Mapping	Evidence/status
FR-05	Link platform profile	The system shall allow a registered user to create and list their own platform-account records.	P1	PlatformAccountController, PlatformAccountService	Implemented in source.
FR-06	Manage linked account	The system shall enforce ownership when a user reads, edits, deletes or synchronizes a platform account.	P1	PlatformAccountService; controller/service tests	Ownership checks implemented; full runtime security audit not performed.
FR-07	Live platform sync	The system shall enable live sync only when the selected platform has a supported live adapter.	P1	PlatformSyncService, adapter registry	LeetCode and Codeforces live sources documented; others described as mock in current README.
FR-08	Persist snapshots	The system shall persist supported synchronization observations with source tags and observation dates.	P1	PlatformSyncPersistenceService, snapshot entity, Flyway	Implemented in source; migrations versioned.
FR-09	Missing metrics	The system shall represent metrics not supplied by a source as null/unavailable and shall not fabricate values.	P1	Platform adapters and stats services	Design documented; source mappings retain unavailable fields.
FR-10	Dashboard summary	The system shall provide owner-scoped dashboard totals from eligible persisted data.	P1	GET /api/dashboard/stats	Implemented in source.
FR-11	Analytics and history	The system shall derive analytics and historical views from stored observations, not synthetic history.	P1	Dashboard analytics/history services	Implemented in source; production data audit not performed.
FR-12	Insights and activity	The system shall provide insight, activity and persisted-data contest-intelligence responses for the authenticated user.	P2	DashboardInsightsService; DashboardActivityContestService	Implemented; this is separate from the external upcoming-contest feed.
FR-13	Leaderboard	The system shall provide authenticated global, college-filtered and personal leaderboard views.	P2	LeaderboardController, LeaderboardService	Implemented in source.
FR-14	Admin-only manual sync	The system shall restrict manual leaderboard synchronization to a principal with ADMIN authority.	P1	SecurityConfig: POST /api/leaderboard/sync	Authorization rule present in source.
FR-15	Public developer profile	The system shall return a sanitized public profile by username without requiring a JWT.	P2	PublicProfileController, PublicProfileService	Implemented in source.
FR-16	Health endpoint	The system shall expose a public process-liveness endpoint.	P1	GET /api/health	Implemented; it is not a database-readiness guarantee.

ID	Feature	Requirement (shall)	Priority	Mapping	Evidence/status
FR-17	Upcoming contest calendar	The system shall return valid upcoming Codeforces contests sorted by start time and capped by the API limit.	P1	ContestCalendarController, ContestCalendarService, CodeforcesContestProvider	23 focused backend tests reported passing; production UI displayed upcoming contests on 11-Oct-2026.
FR-18	Contest cache and resilience	The system shall cache contest results for a configured TTL and serve stale cached data when a refresh fails, where stale data exists.	P1	ContestCalendarService and configuration	Source/tests included in contest-calendar test suite; no separate load test performed.
FR-19	Contest UI states	The frontend shall show upcoming contests with local-time display, duration, countdown, official links, filter/refresh controls, and loading/empty/error states.	P1	UpcomingContestSection, useContestCalendar	6 frontend contest tests and Vite production build reported passing; main UI observed in production.
FR-20	Fixture mode	The contest provider shall support deterministic fixture scenarios without outbound Codeforces requests.	P2	FixtureContestProvider, fixture mode docs/tests	Fixture scenario tests included; production default is Codeforces unless configured otherwise.

5. Non-functional Requirements

ID	Quality area	Requirement	Priority	Current evidence / verification gap
NFR-01	Authentication security	Passwords shall be verified against stored password hashes, and JWT signing shall use a configured runtime secret.	P1	Source/configuration supports this; independent penetration test not performed.
NFR-02	Stateless API access	Protected API resources shall require a valid bearer token and shall not depend on an HTTP session.	P1	Spring Security configured for stateless sessions.
NFR-03	Authorization and ownership	A user shall not access another user's private platform-account or dashboard data by changing a record identifier.	P1	Owner-scoped service operations are implemented; security regression tests should remain in CI.
NFR-04	Secret management	Database passwords and JWT secrets shall not be committed or embedded in frontend assets.	P1	Runtime environment configuration is used.
NFR-05	Schema integrity	Database schema changes shall be versioned; migrations shall validate and automatic Hibernate schema mutation shall be disabled.	P1	Flyway and Hibernate configuration implement this policy.

ID	Quality area	Requirement	Priority	Current evidence / verification gap
NFR-06	Safe error responses	HTTP responses shall not expose stack traces, exception details, credentials or raw request bodies.	P1	Error response configuration and safe auth diagnostics are present; runtime log review still required.
NFR-07	CORS	Production browser origins shall be explicitly allowlisted; wildcard origins shall not be accepted in production.	P1	Exact-origin CORS configuration present; production environment must supply correct values.
NFR-08	Data provenance	Persisted metrics and contest responses shall identify a source and timestamps where available.	P1	Source tags and fetch/update timestamps are part of the data model/DTOs.
NFR-09	Upstream resilience	Upstream failures shall not be silently converted into mock metrics; unavailable fields shall remain unavailable.	P1	Adapter error handling and documented platform-data policy.
NFR-10	Contest refresh efficiency	The upcoming contest provider shall use a configurable TTL cache to limit repetitive upstream fetches.	P1	Default TTL is PT10M in the current configuration; no formal throughput benchmark performed.
NFR-11	Contest response validation	The service shall filter malformed, non-upcoming or incomplete contest records before returning them to clients.	P1	Codeforces provider applies phase, ID, name, time and duration validation.
NFR-12	Accessibility and responsive layout	The UI should remain usable on common desktop and mobile viewport sizes with readable states and operable controls.	P2	Responsive styling is present; formal accessibility audit and cross-device test suite not evidenced.
NFR-13	Performance target	For planning, typical authenticated API calls should target p95 <= 2 seconds excluding cold start and upstream provider latency; contest refreshes may be bounded by upstream timeout.	P2	Proposed target; no production load test or SLO measurement has been completed.
NFR-14	Availability and recovery	The deployed service should provide health monitoring, bounded upstream timeouts and observable failures.	P1	Health endpoint exists; hosting health checks/logs are available, but formal uptime guarantee is not specified.

6. Data Requirements

6.1 Principal data entities

Entity / record	Data and relationship requirements
User	Unique email; stores account identity and authentication-related fields. Password material must not be exposed in API responses.
Profile	At most one profile per user; profile username is unique; profile fields are validated and length-limited.
PlatformAccount	Owned by one user; unique constraint on (user_id, platform); stores platform/handle/profile URL and synchronization state.
PlatformStatSnapshot	Dated metric observation associated with user/platform; unique key documented as (user_id, platform, snapshot_date); source tag and timestamps retained. Metrics are nullable.
Leaderboard cache/record	Stores or serves derived leaderboard information; must reflect eligibility/source rules and owner boundaries.
Contest DTO/cache item	Contest identifier, platform, name, start time, duration, official URL, source and fetched-at timestamp; data is fetched from provider/cache rather than persisted as a user history snapshot in the current feature.

6.2 Data validation rules

- Request DTOs shall apply Jakarta validation to required fields and format/length constraints.
- The API shall reject invalid contest limits outside 1–100 and platform filters other than ALL or CODEFORCES.
- The contest provider shall accept only valid upcoming records with a positive ID, nonblank name, future start time and positive duration.
- Null metrics shall mean unavailable/not supplied; null shall not be interpreted as zero unless a specific metric contract explicitly defines that behavior.
- Database foreign keys and uniqueness constraints shall protect entity relationships and prevent duplicate user/platform/snapshot keys.

7. Security, Privacy and Compliance

- Registration and login, GET /api/health, and public-profile GET requests are the only intentionally public API families documented by the security configuration. Other APIs require authentication; manual leaderboard sync requires ADMIN authority.
- Account-level read, update, delete and sync operations shall check that the authenticated owner matches the requested account.
- Public developer profile responses shall use a sanitized DTO rather than serializing internal user or credential entities.
- Login and registration requests shall not attach a stored JWT; protected requests shall continue to use bearer authentication. Request diagnostics may include request ID, operation, status and durations, but not credentials, request bodies, JWTs, authorization headers or exception messages.
- Database and signing secrets shall be injected by runtime configuration/secret management. Frontend Vite variables are public and shall not contain secrets.

- Public pages and undocumented endpoints do not imply permission for automated data collection. The LeetCode endpoint's contractual stability and terms remain a limitation. Use each upstream provider only under applicable authorization and terms.
- The system does not implement a separate mentor/college-admin role workflow, consent-based monitoring, student inactivity monitoring, or campus-wide institutional controls.

Data integrity rule
Do not label synthetic/mock values as verified live statistics. UI source labels and persisted source tags must remain consistent with the provider that actually produced the data.

8. Use Cases

UC-01 — Register and sign in

Primary actor: Visitor. Preconditions: application and API available. Main flow: visitor submits valid registration data; server creates the account; visitor submits credentials; server verifies them and returns a JWT/user summary. Alternatives: invalid input, duplicate email or invalid credentials are rejected with safe errors. Postcondition: the client can access protected routes.

UC-02 — Connect and synchronize a platform account

Primary actor: Registered user. Preconditions: user is authenticated. Main flow: user submits platform and handle; server stores an owned account; user requests sync; server checks ownership and live-source support, invokes an adapter, records outcome and exposes status. Alternatives: unsupported platform, account not found, rate limit, timeout or upstream outage. Postcondition: available stats are tagged with a source; unavailable metrics remain null.

UC-03 — Review personal progress

Primary actor: Registered user. Preconditions: JWT available; zero or more platform snapshots exist. Main flow: client requests dashboard, analytics, history and insights; services compute views from eligible stored observations. Alternative: with no snapshots, unavailable results remain empty/null rather than fabricated. Postcondition: the user sees traceable progress data.

UC-04 — View public developer profile

Primary actor: Public visitor. Preconditions: a username maps to public profile data. Main flow: visitor opens /u/{username}; frontend requests the public-profile endpoint without JWT; server returns sanitized public fields. Alternative: unknown username returns a not-found response. Postcondition: only the public DTO is presented.

UC-05 — Browse upcoming contests

Primary actor: Authenticated user. Preconditions: frontend and /api/contests endpoint available. Main flow: page requests the contest calendar; backend returns cached/fresh Codeforces contest DTOs; UI displays local start time, duration, countdown and official link. User may refresh or filter. Alternatives: empty feed displays empty state; provider outage can return cached stale items with source status, or an error state when no usable cache exists. Postcondition: the user can navigate to the official contest page.

UC-06 — View leaderboard

Primary actor: Registered user. Preconditions: valid JWT. Main flow: user requests global, college-filtered or personal rank results; backend applies pagination/filter/sort behavior. Admin-only alternative: only an ADMIN principal may trigger manual sync. Postcondition: results reflect the current leaderboard service data.

9. Verification and Acceptance Criteria

Acceptance is based on repeatable tests and runtime checks. Test counts below are evidence reported for the current feature work on 11 October 2026; they are not a statement that every product requirement has an independent test.

ID	Acceptance criterion	Verification method	Current status
AC-01	Valid registration/login succeeds; invalid inputs and credentials are safely rejected.	Auth controller/service tests; deployed smoke test.	Auth diagnostics focused tests previously passed; full end-to-end negative/positive matrix should be run in release verification.
AC-02	One user cannot access another user's account records by substituting IDs.	Automated ownership/security tests and JWT-authenticated API checks.	Source enforces owner-scoped services; complete production security test not reported.
AC-03	Platform sync shows real source tags and does not fill absent metrics with invented numbers.	Adapter and persistence tests; inspect source tags in UI/API.	LeetCode and Codeforces live adapters documented; unsupported platforms shown as mock/unavailable per configured source.
AC-04	Contest calendar backend scenarios cover valid items, malformed items, cache, stale fallback and failures.	Focused backend tests.	23 passed, 0 failures/errors/skips reported.
AC-05	Contest calendar frontend tests pass for transformation/state behavior.	Node test runner.	6 passed, 0 failures reported.
AC-06	Frontend builds successfully from the exact pushed main commit.	Vercel production build logs.	Passed on commit 9061773 on 11-Oct-2026; fresh build without cache completed and deployment completed.
AC-07	Upcoming Codeforces cards appear in production with date, local timezone, countdown, filter and official link.	Production browser smoke test.	Production screenshot showed four upcoming cards, local timezone, filter and refresh controls; continuous countdown/filter interactions still require manual interaction testing.
AC-08	Flyway migrates and validates schema against a real MySQL 8 instance.	Integration test against MySQL 8 with Docker/MySQL available.	Required release check; not established by the contest-specific test result.
AC-09	Public and protected routes enforce the configured policy.	Test public health/profile access and protected routes with absent, invalid and valid JWTs.	Required release check; not completely verified by the supplied production screenshot.
AC-10	No credentials or token values appear in diagnostics.	Review auth diagnostic output and perform a controlled log inspection.	Diagnostic code was reviewed to avoid sensitive values; routine log sampling remains an operational check.

10. Requirements Traceability

The following compact matrix maps requirement areas to primary implementation locations and tests. Individual feature-level source mappings are provided in Sections 4 and 5.

Requirement area	Primary implementation	Main verification artefacts
Authentication / identity	AuthController, AuthService, JwtService, SecurityConfig	AuthControllerTest, AuthServiceTest, auth request-header tests
Profile	ProfileController, ProfileService, PublicProfileController	Profile controller/service tests; public profile tests
Platform accounts / ownership	PlatformAccountController, PlatformAccountService, PlatformSyncService	Platform account controller/service tests, sync/adapter tests
Live platform statistics	Platform adapter registry, LeetCode and Codeforces adapters	LeetCodeClientTest/adapter tests, Codeforces adapter tests
Persistence / history	PlatformSyncPersistenceService, PlatformStatSnapshot, Flyway migrations	Persistence and history tests; MySQL migration integration test
Dashboard / analytics	DashboardController and dashboard services	Dashboard analytics/history/insights/activity service tests
Leaderboard	LeaderboardController, LeaderboardService, SecurityConfig	Leaderboard tests and admin-authorization tests
Contest calendar backend	ContestCalendarController, ContestCalendarService, CodeforcesContestProvider, FixtureContestProvider	ContestCalendarControllerTest, ContestCalendarServiceTest, CodeforcesContestProviderTest, ContestCalendarFixtureScenarioTest
Contest calendar UI	UpcomingContestSection, useContestCalendar, contestCalendarApi.ts	frontend/tests/contestCalendar.test.mjs; Vite production build
Deployment / operations	application.yml, application-prod.yml, Dockerfile, frontend/vercel.json	Build logs, health probe, environment review, production smoke tests

11. Constraints, Assumptions and Out of Scope

11.1 Assumptions

- The user has network access to the deployed frontend and API.
- Production backend environment variables and database connectivity are configured outside the frontend repository.
- The upstream Codeforces public API remains available and provides contest metadata in its documented response schema.
- Platform integrations may provide incomplete fields, and provider timestamps/timezone conversion must be handled explicitly.

11.2 Known limitations and risks

- Free hosting tiers can cold-start; response latency can include host wake-up and upstream API time. No service-level availability guarantee is stated.
- LeetCode uses an undocumented endpoint; changes can break sync independently of KodBTW releases.
- The current README labels CodeChef, GeeksforGeeks and HackerRank adapters as MOCK. A mock label is not equivalent to live data or verified account ownership.
- The contest calendar currently supports Codeforces only; the platform filter is not a promise of multiple live contest providers.
- The contest calendar is fetched/cached in memory; it is not a permanent contest-history archive.
- Production screenshots verify the visible contests page at one moment, not long-term uptime, mobile accessibility, or sustained correctness of countdown ticks.

11.3 Explicitly out of scope unless separately approved

- Campus or institution administrator dashboards, college registration workflows and mentor permissions.
- Automated student surveillance, inactivity detection, consent and intervention workflows.
- AI-driven DSA topic-gap analysis, AI suggestions, workshop recommendations or personalized roadmaps.
- Account ownership verification with the external coding platforms.
- Official/authorized CodeChef, GeeksforGeeks and HackerRank live sync until a permitted and stable source is approved and implemented.
- Native mobile apps, push notifications, reminders and a permanent cross-platform contest history store.

Appendix A. Deployment and Runtime Configuration

A.1 Required backend settings

Variable	Purpose	Requirement
DB_URL	JDBC URL for MySQL	Required
DB_USERNAME	Database account	Required; least privilege
DB_PASSWORD	Database credential	Required; use host secret store
JWT_SECRET	JWT HMAC signing key	Required; generate strong random secret
JWT_EXPIRATION_MS	Token lifetime in milliseconds	Defaults to 86400000 (24h) in the current configuration
CORS_ALLOWED_ORIGINS	Comma-separated exact origins	Must be configured for deployed frontend in production
PORT	HTTP listen port	Defaults to 8080; host may supply the value
VITE_API_BASE_URL	Public frontend API origin at build time	Must point to deployed API when cross-origin routing is used; never store secrets here
CONTEST_CALENDAR_PROVIDER	Contest provider selector	Defaults to codeforces; fixture is for deterministic testing
CONTEST_CALENDAR_CACHE_TTL	Contest cache duration	Defaults to PT10M
CONTEST_CALENDAR_FIXTURE_SCENARIO	Fixture scenario	Supports multiple, empty, unavailable, stale when fixture provider selected

A.2 Release workflow

1. Run focused tests for changed backend/frontend features and run the full backend and frontend suites when practical.
2. Build backend and frontend from the intended commit; verify migrations against MySQL 8 with the release environment.
3. Push an isolated commit; verify the Vercel build uses that commit and completes successfully.
4. Verify the Render backend is running the corresponding compatible version and `/api/health` returns its documented liveness response.
5. Perform authenticated smoke tests for login, GET /api/auth/me, one protected dashboard API, and GET /api/contests. Do not share passwords, JWTs, cookies or response payloads containing personal information in debugging artifacts.
6. For contest calendar verification, inspect source status, freshness time, local-time conversion, filter behavior, countdown updates, refresh behavior, stale-data behavior and the empty/error states.

Appendix B. Acronyms and References

Reference	Description
IEEE Std 830-1998	Historical SRS structure reference; this document is adapted and not formally certified.
KodBTW repository	https://github.com/sajalgoyal007/KodBTW
Frontend production URL	https://kodbtw.vercel.app
Backend service URL	https://kodbtw-backend.onrender.com
Codeforces API help	https://codeforces.com/apiHelp
Architecture notes	docs/ARCHITECTURE.md
Codeforces integration notes	docs/CODEFORCES_INTEGRATION.md
Platform data acquisition notes	docs/platform-data-acquisition.md
Contest fixture-mode notes	docs/contest-calendar-fixture-mode.md
Provided baseline SRS	SRS.pdf, Version 1.0; six pages, snapshot 1d45df1; used as a starting point, not as an authority where it conflicted with current main.

End of specification

Requirement statuses are tied to the evidence listed in this document. A source implementation, passing unit test, successful frontend build and successful production interaction are distinct forms of evidence and must not be treated as interchangeable.